

RESUME SKILL EXTRACTION & JOB MATCHING SYSTEM

Submitted by:

SIVANI J BIJU

24UBC154

III BCA - A

1. Abstract

Resume screening can be time-consuming when recruiters need to analyze many applications. This project develops a Resume Skill Extraction & Job Matching System using Natural Language Processing (NLP). The system extracts technical skills from resumes, compares them with job descriptions, and calculates similarity using TF-IDF and Cosine Similarity. It also performs candidate ranking, skill comparison, and model evaluation. A Flask-based web interface was developed for PDF resume analysis and deployed using Render. The system achieved 100% Precision, 98.73% Recall, and 99.36% F1-score on the evaluation dataset.

2. Introduction

Resume screening is an important part of recruitment, but manually analyzing multiple resumes can be time-consuming. Natural Language Processing (NLP) can automate this process by extracting useful information from resume text.

This project develops a Resume Skill Extraction & Job Matching System that extracts technical skills from resumes and compares them with job descriptions. TF-IDF and Cosine Similarity are used to calculate similarity, while candidate ranking and skill comparison help identify suitable candidates. A Flask web interface was also developed for PDF resume analysis.

3. Problem Statement

Recruiters often need to compare many resumes with job requirements. Manual comparison requires considerable time and may result in inconsistent evaluation.

The proposed system addresses this problem by:

- Extracting skills automatically from resumes.
- Comparing resume skills with job requirements.
- Calculating resume-job similarity.
- Ranking candidates according to their matching scores.
- Identifying matched and missing skills.

4. Objectives

The main objectives are:

1. To preprocess resume and job-description text using NLP.
2. To extract relevant technical skills.
3. To calculate similarity using TF-IDF and Cosine Similarity.
4. To rank candidates according to job relevance.
5. To identify matched and missing skills.
6. To evaluate the skill extraction performance.
7. To provide a simple web interface for resume analysis.

5. Dataset Description

The project uses three main files:

<u>File</u>	<u>Description</u>
resumes.csv	Contains candidate IDs, names, and resume text
job_descriptions.csv	Contains job IDs, job titles, and job descriptions
skills_vocabulary.txt	Contains the technical skills used for extraction

The datasets contain sample resumes and job descriptions representing different technical roles.

Resume dataset:

- resume_id
- candidate_name
- resume_text

Job dataset:

- jd_id
- job_title

- job_text

6. Technologies Used

The project was implemented using:

- Python , NLTK , Scikit-learn , Pandas , NumPy , Matplotlib , Flask , PyPDF,
HTML/CSS , GitHub , Render

7. System Methodology

The system follows the following workflow:

Resume / Job Description → Text Preprocessing → Skill Extraction → TF-IDF → Cosine Similarity → Job Matching → Candidate Ranking → Evaluation

Main stages:

1. Load the datasets.
2. Preprocess the text.
3. Extract skills using the skill vocabulary.
4. Convert text into TF-IDF vectors.
5. Calculate Cosine Similarity.
6. Compare resumes with job descriptions.
7. Rank candidates.
8. Compare matched and missing skills.
9. Evaluate the skill extraction system.

8. Text Preprocessing

Text preprocessing prepares resume and job-description text for analysis.

The preprocessing steps include:

- Converting text to lowercase.
- Removing unwanted characters using regular expressions.
- Removing stopwords using NLTK.
- Preparing cleaned text for further NLP processing.

For example:

Original text: The candidate has strong skills in Python and Machine Learning.

Processed text: candidate strong skills python machine learning

Preprocessing helps reduce unnecessary words and improves text similarity calculation.

Text Preprocessing Demonstration

4. Text Preprocessing

Text preprocessing is an important step in the NLP pipeline. It converts raw resume and job-description text into a cleaner form for further analysis.

Preprocessing Steps

1. Convert text to lowercase
2. Remove unnecessary characters
3. Remove punctuation
4. Remove stopwords
5. Normalize the text

Purpose

The preprocessing step reduces unnecessary variations in the text and prepares the content for skill extraction and similarity calculation.

```
# Preprocessing Demonstration
import nltk
nltk.download("stopwords", quiet=True)
from source_code import preprocess_text
```

The preprocessing step reduces unnecessary variations in the text and prepares the content for skill extraction and similarity calculation.

```
# Preprocessing Demonstration
import nltk
nltk.download("stopwords", quiet=True)
from source_code import preprocess_text

sample_text = resumes_df.iloc[0]["resume_text"]

print("ORIGINAL TEXT:")
print(sample_text)

print("\n" + "="*70)

print("\nPREPROCESSED TEXT:")
print(preprocess_text(sample_text))

... ORIGINAL TEXT:
Experienced data scientist with 3 years in Python, Machine Learning, and Deep Learning. Skilled in Pandas, NumPy, Scikit-learn, and TensorFlow. Built predictive models for customer churn worked sql database

=====
PREPROCESSED TEXT:
experienced data scientist years python machine learning deep learning skilled pandas numpy scikit learn tensorflow built predictive models customer churn worked sql database
```

9. Skill Extraction

The system extracts technical skills using a predefined skills vocabulary.

The vocabulary contains skills such as:

- Python , Machine Learning , Deep Learning , TensorFlow , PyTorch , NLP ,
BERT , AWS , Docker , Statistics

The system searches the processed resume text for vocabulary terms and returns the detected skills.

Example:-

For a resume containing: Python, Machine Learning, Deep Learning and TensorFlow

the system extracts: Python, Machine Learning, Deep Learning, TensorFlow

Skill Extraction Demonstration

5. Skill Extraction

The skill extraction module identifies relevant technical skills from resume text using a predefined skill vocabulary.

Extraction Process

1. Load the predefined skill vocabulary.
2. Preprocess the resume text.
3. Compare the resume text with the vocabulary.
4. Identify matching skills.
5. Display the extracted skills.

Purpose

Skill extraction helps convert unstructured resume text into structured information that can be used for job matching and candidate comparison.

```
# Skill Extraction Demonstration

from source_code import extract_skills, load_skills_vocabulary

skills_vocab = load_skills_vocabulary(
    "Dataset/skills_vocabulary.txt"
)
```

Purpose

Skill extraction helps convert unstructured resume text into structured information that can be used for job matching and candidate comparison.

```
# Skill Extraction Demonstration

from source_code import extract_skills, load_skills_vocabulary

skills_vocab = load_skills_vocabulary(
    "Dataset/skills_vocabulary.txt"
)

extracted_skills = extract_skills(
    sample_text,
    skills_vocab
)

print("\n EXTRACTED SKILLS:")
print(", ".join(extracted_skills))

... EXTRACTED SKILLS:
python, sql, machine learning, deep learning, tensorflow, scikit-learn, pandas, numpy, natural language processing, tf-idf, statistics, data visualization, matplotlib, seaborn
```

10. TF-IDF and Cosine Similarity

TF-IDF (Term Frequency–Inverse Document Frequency) represents text numerically by assigning importance to words.

The project applies TF-IDF to the preprocessed resume and job description.

The similarity between them is calculated using Cosine Similarity.

The value ranges from 0 to 1:

- 0 → very little similarity
- 1 → very high similarity

The similarity is converted into a percentage for easier interpretation.

Result:-

The demonstration produced:

Cosine Similarity Score: 31.85%

TF-IDF Vocabulary Size: 58

TF-IDF and Cosine Similarity Result

6. TF-IDF & Cosine Similarity

TF-IDF Vectorization

TF-IDF (Term Frequency–Inverse Document Frequency) converts text into numerical vectors based on the importance of words within the documents.

Cosine Similarity

Cosine Similarity measures how similar two text vectors are. In this project, it is used to compare a resume with a job description.

Process

Resume + Job Description

↓

TF-IDF Vectorization

↓

Numerical Vectors

↓

Cosine Similarity

↓

Similarity Score (%)

Purpose

A higher similarity score indicates that the resume has greater textual similarity with the selected job description.

```
# TF-IDF + Cosine Similarity Demonstration

# TF-IDF + Cosine Similarity Demonstration

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# Preprocess both texts using the project's preprocessing function
resume_processed = preprocess_text(resume_text)
job_processed = preprocess_text(job_text)

# Convert the preprocessed texts into TF-IDF vectors
vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform([resume_processed, job_processed])

# Calculate cosine similarity
similarity_score = cosine_similarity(
    tfidf_matrix[0:1],
    tfidf_matrix[1:2]
)[0][0] * 100

print("Resume Text:")

[ resume_processed, job_processed ]

# Calculate cosine similarity
similarity_score = cosine_similarity(
    tfidf_matrix[0:1],
    tfidf_matrix[1:2]
)[0][0] * 100

print("Resume Text:")
print(resume_text)

print("\nJob Description:")
print(job_text)

print("\nTF-IDF Vocabulary Size:", len(vectorizer.get_feature_names_out()))
print(f"\nCosine Similarity Score: {similarity_score:.2f}%")

... Resume Text:
Experienced data scientist with 3 years in Python, Machine Learning, and Deep Learning. Skilled in Pandas, NumPy, Scikit-learn, and TensorFlow. Built predictive models for cu

Job Description:
We are looking for a Machine Learning Engineer with strong skills in Python, TensorFlow or PyTorch, and Deep Learning. Experience with Natural Language Processing, text class

TF-IDF Vocabulary Size: 58
Cosine Similarity Score: 31.85%
```

11. Job Matching

The system compares a resume against available job descriptions.

For the selected resume, the results were:

<u>Job ID</u>	<u>Job Title</u>	<u>Match Score</u>
JD001	Machine Learning Engineer	31.85%
JD003	NLP Engineer	11.59%
JD002	Backend Software Engineer	1.41%

The Machine Learning Engineer position produced the highest similarity score.

Job Matching Results

7. Job Matching

The job-matching module compares a resume with multiple job descriptions and calculates a similarity score for each job.

Matching Process

Resume
↓
Compare with Job Descriptions
↓
Calculate Cosine Similarity
↓
Generate Match Scores
↓
Identify Best-Matching Job

Output

The system displays the similarity score for each available job description.

A higher score indicates a stronger match between the resume and the job description.

Purpose

This module helps candidates identify job roles that are most relevant to their skills and experience.

Purpose

This module helps candidates identify job roles that are most relevant to their skills and experience.

[+ Code](#) [+ Text](#)

```
# Job Matching Results
# Job Matching Results

job_results = []

for _, row in jobs_df.iterrows():
    job_text_current = row["job_text"]

    # Preprocess resume and job description
    resume_processed = preprocess_text(resume_text)
    job_processed = preprocess_text(job_text_current)

    # TF-IDF Vectorization
    vectorizer = TfidfVectorizer()
    tfidf_matrix = vectorizer.fit_transform(
        [resume_processed, job_processed]
    )

    # Calculate cosine similarity
    score = cosine_similarity(
        tfidf_matrix[0:1],
        tfidf_matrix[1:2]
    )[0][0] * 100
```

```
# Calculate cosine similarity
score = cosine_similarity(
    tfidf_matrix[0:1],
    tfidf_matrix[1:2]
)[0][0] * 100

job_results.append({
    "Job ID": row["jd_id"],
    "Job Title": row["job_title"],
    "Match Score (%)": round(score, 2)
})

job_results_df = pd.DataFrame(job_results)

# Sort by highest match score
job_results_df = job_results_df.sort_values(
    by="Match Score (%)",
    ascending=False
).reset_index(drop=True)

display(job_results_df)
```

	Job ID	Job Title	Match Score (%)
0	JD001	Machine Learning Engineer	31.85
1	JD003	NLP Engineer	11.59
2	JD002	Backend Software Engineer	1.41

12. Candidate Ranking

The system compares all available resumes with the selected job description and ranks candidates according to similarity.

For the Machine Learning Engineer position:

<u>Rank</u>	<u>Candidate</u>	<u>Score</u>
1	Arjun Menon	53.28%
2	Aditi Sharma	31.85%
3	Meera Krishnan	16.67%
4	Karthik Pillai	14.99%
5	Vikram Rao	10.96%
6	Rahul Verma	7.22%
7	Priya Iyer	6.26%
8	Sneha Nair	4.85%

Arjun Menon achieved the highest similarity score.

Candidate Ranking Graph

8. Candidate Ranking

The candidate-ranking module evaluates all resumes against a selected job description and ranks candidates according to their similarity scores.

Ranking Process

Select Job Role

↓

Compare All Resumes

↓

Calculate Match Scores

↓

Sort Candidates by Score

↓

Generate Candidate Ranking

Result

Candidates are arranged from the **highest match score to the lowest match score**.

The candidate with the highest score is considered the strongest textual match for the selected job description.

```
def ranking_results(candidate_name, score):
    """Returns a dictionary with candidate name and match score"""
    return {"Candidate": candidate_name,
            "Match Score (%)": round(score, 2)}

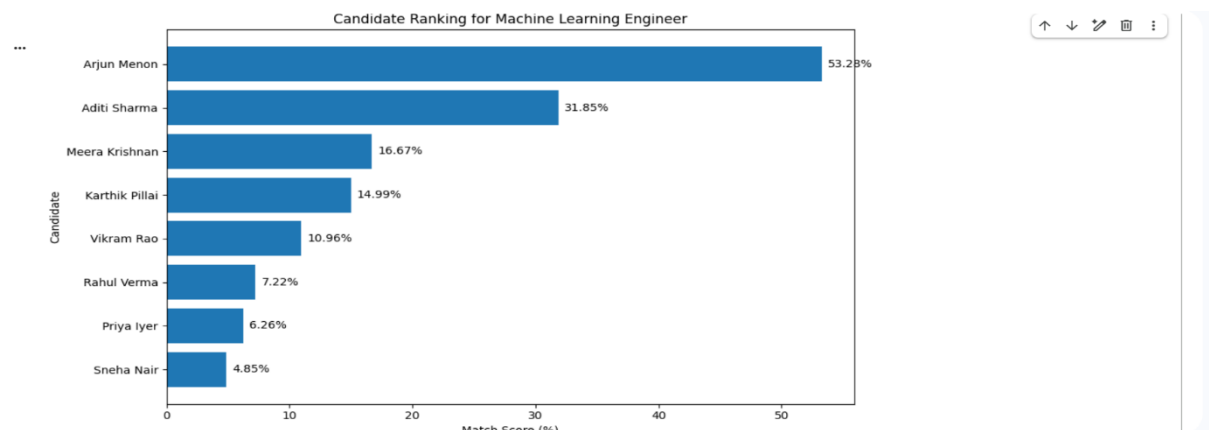
# Create a list of dictionaries for each candidate
ranking_results_list = [
    {"Candidate": "Arjun Menon", "Match Score (%)": 53.28},
    {"Candidate": "Aditi Sharma", "Match Score (%)": 31.85},
    {"Candidate": "Meera Krishnan", "Match Score (%)": 16.67},
    {"Candidate": "Karthik Pillai", "Match Score (%)": 14.99},
    {"Candidate": "Vikram Rao", "Match Score (%)": 10.96},
    {"Candidate": "Rahul Verma", "Match Score (%)": 7.22},
    {"Candidate": "Priya Iyer", "Match Score (%)": 6.26},
    {"Candidate": "Sneha Nair", "Match Score (%)": 4.85}
]

# Create a pandas DataFrame from the list
ranking_df = pd.DataFrame(ranking_results_list)

# Sort candidates by highest match score
ranking_df = ranking_df.sort_values(
    by="Match Score (%)",
    ascending=False
).reset_index(drop=True)

display(ranking_df)
```

	Candidate	Match Score (%)
0	Arjun Menon	53.28
1	Aditi Sharma	31.85
2	Meera Krishnan	16.67
3	Karthik Pillai	14.99
4	Vikram Rao	10.96
5	Rahul Verma	7.22
6	Priya Iyer	6.26
7	Sneha Nair	4.85



13. Skill Comparison

The system also compares the skills in a candidate's resume with the skills required for a job.

For Aditi Sharma applying for the Machine Learning Engineer position:

Matched Skills:

- Deep Learning
- Machine Learning
- Natural Language Processing
- Python
- Statistics
- TensorFlow

Missing Skills:

- AWS
- BERT
- Docker
- Machine Learning Pipelines
- PyTorch

This comparison helps identify the candidate's strengths and skill gaps.

Skill Comparison Results

9. Skill Comparison & Skill Gap Analysis

The skill comparison module compares the skills extracted from a candidate's resume with the skills required for a selected job description.

Comparison

The system identifies:

- ✔ **Matched Skills** — skills present in both the resume and job description.
- ✘ **Missing Skills** — skills required by the job but not found in the resume.

Process

Job Required Skills + Candidate Skills

↓

Compare Skill Sets

↓

Matched Skills + Missing Skills

↓

Skill Gap Analysis

Purpose

This analysis helps identify how well a candidate's skills satisfy the requirements of a particular job and highlights areas where additional

Purpose

This analysis helps identify how well a candidate's skills satisfy the requirements of a particular job and highlights areas where additional skills may be needed.

```
# Skill Comparison Results

job_skills = set(
    extract_skills(
        selected_job["job_text"],
        skills_vocab
    )
)

candidate = resumes_df.iloc[0]

resume_skills = set(
    extract_skills(
        candidate["resume_text"],
        skills_vocab
    )
)

matched_skills = sorted(
    job_skills.intersection(resume_skills)
)

missing_skills = sorted(

matched_skills = sorted(
    job_skills.intersection(resume_skills)
)

missing_skills = sorted(
    job_skills.difference(resume_skills)
)

print("📄 JOB:", selected_job["job_title"])
print("👤 CANDIDATE:", candidate["candidate_name"])

print("\n✔️ MATCHED SKILLS:")
print(" ", ".join(matched_skills))

print("\n✘ MISSING SKILLS:")
print(" ", ".join(missing_skills))

📄 JOB: Machine Learning Engineer
👤 CANDIDATE: Aditi Sharma

✔️ MATCHED SKILLS:
deep learning, machine learning, natural language processing, python, statistics, tensorflow

✘ MISSING SKILLS:
aws, bert, docker, machine learning pipelines, pytorch
```

14. Model Evaluation

The skill extraction system was evaluated using manually identified expected skills and extracted skills.

The evaluation produced the following results:

Metric Result

Precision 100%

Recall 98.73%

F1-Score 99.36%

The system produced 78 true positives, 0 false positives, and 1 false negative.

Interpretation:

The 100% precision indicates that the extracted skills were relevant and did not contain false-positive skills in the evaluation dataset.

The 98.73% recall shows that almost all expected skills were successfully detected.

The 99.36% F1-score indicates an excellent balance between precision and recall.

Model Evaluation Output

 11. Model Evaluation

The performance of the skill extraction system is evaluated using standard classification metrics.

 Evaluation Metrics

Metric	Description
Precision	Measures how many extracted skills are actually relevant.
Recall	Measures how many relevant skills were successfully extracted.
F1-Score	Provides a combined measure of precision and recall.

 Purpose

Evaluation helps determine how accurately the system identifies skills from resumes and provides an objective measure of its performance.

```
# Run Evaluation
import subprocess

result = subprocess.run(
    ["python", "evaluate.py"],
    capture_output=True,
    text=True
)

print(result.stdout)
```

SKILL EXTRACTION EVALUATION

Candidate: Aditi Sharma (R001)

Actual Skills: 14

Extracted Skills: 14

True Positives: 14

False Positives: 0

False Negatives: 0

Precision: 100.00%

Recall: 100.00%

F1-Score: 100.00%

Candidate: Rahul Verma (R002)

Actual Skills: 12

Extracted Skills: 12

True Positives: 12

False Positives: 0

False Negatives: 0

Precision: 100.00%

Recall: 100.00%

F1-Score: 100.00%

Candidate: Sneha Nair (R003)

Actual Skills: 9

Extracted Skills: 8

True Positives: 8

False Positives: 0

False Negatives: 1

Candidate: Sneha Nair (R003)
Actual Skills: 9
Extracted Skills: 8
True Positives: 8
False Positives: 0
False Negatives: 1
Precision: 100.00%
Recall: 88.89%
F1-Score: 94.12%

Candidate: Arjun Menon (R004)
Actual Skills: 12
Extracted Skills: 12
True Positives: 12
False Positives: 0
False Negatives: 0
Precision: 100.00%
Recall: 100.00%
F1-Score: 100.00%

Candidate: Priya Iyer (R005)
Actual Skills: 1
Extracted Skills: 1
True Positives: 1
False Positives: 0
False Negatives: 0
Precision: 100.00%
Recall: 100.00%
F1-Score: 100.00%

Candidate: Vikram Rao (R006)
Actual Skills: 9

Candidate: Vikram Rao (R006)
Actual Skills: 9
Extracted Skills: 9
True Positives: 9
False Positives: 0
False Negatives: 0
Precision: 100.00%
Recall: 100.00%
F1-Score: 100.00%

Candidate: Meera Krishnan (R007)
Actual Skills: 11
Extracted Skills: 11
True Positives: 11
False Positives: 0
False Negatives: 0
Precision: 100.00%
Recall: 100.00%
F1-Score: 100.00%

Candidate: Karthik Pillai (R008)
Actual Skills: 11
Extracted Skills: 11
True Positives: 11
False Positives: 0
False Negatives: 0
Precision: 100.00%
Recall: 100.00%
F1-Score: 100.00%

OVERALL EVALUATION

Total True Positives: 78
Total False Positives: 0
Total False Negatives: 1

Overall Precision: 100.00%
Overall Recall: 98.73%
Overall F1-Score: 99.36%

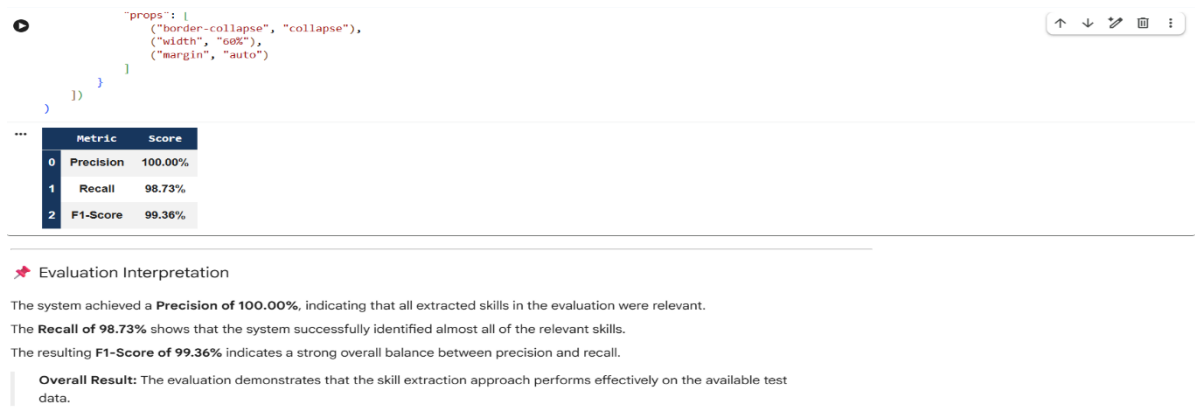
Evaluation completed successfully.

Professional Evaluation Summary

```
evaluation_summary = pd.DataFrame([
    {
        "Metric": [
            "Precision",
            "Recall",
            "F1-Score"
        ],
        "Score": [
            "100.00%",
            "98.73%",
            "99.36%"
        ]
    }
])

display(

display(
    evaluation_summary.style
        .set_properties(**{
            "text-align": "center",
            "padding": "10px",
            "font-size": "14px"
        })
        .set_table_styles([
            {
                "selector": "th",
                "props": [
                    ("background-color", "#17365D"),
                    ("color", "white"),
                    ("font-weight", "bold"),
                    ("text-align", "center")
                ]
            },
            {
                "selector": "td",
                "props": [
                    ("text-align", "center"),
                    ("font-weight", "bold")
                ]
            }
        ])
        .set_table_styles([
            {
                "selector": "table",
                "props": [
                    ("border-collapse", "collapse"),
                    ("width", "60%"),
                    ("margin", "auto")
                ]
            }
        ])
    )
)
```



15. Implementation

The project was implemented in Python using separate modules and functions.

Important components include:

Text Preprocessing: The `preprocess_text()` function cleans and prepares text for NLP processing.

Skill Extraction: The `extract_skills()` function searches the processed text using the predefined skill vocabulary.

Similarity Calculation: TF-IDF vectors are generated using Scikit-learn and compared using Cosine Similarity.

Candidate Ranking: Each resume is compared with the selected job description and sorted according to similarity score.

Evaluation: The evaluation program calculates:

- True Positives
- False Positives
- False Negatives
- Precision
- Recall
- F1-score

16. Flask Web Interface

A Flask-based web interface was developed to provide a simple way to analyze resumes.

The user can:

1. Upload a PDF resume.
2. Enter the job title.
3. Enter the job description.
4. Click Analyze Resume.
5. View the matching results.

The interface displays:

- Match Score , Skill Coverage , Matched Skills , Missing Skills ,Extracted Resume Skills

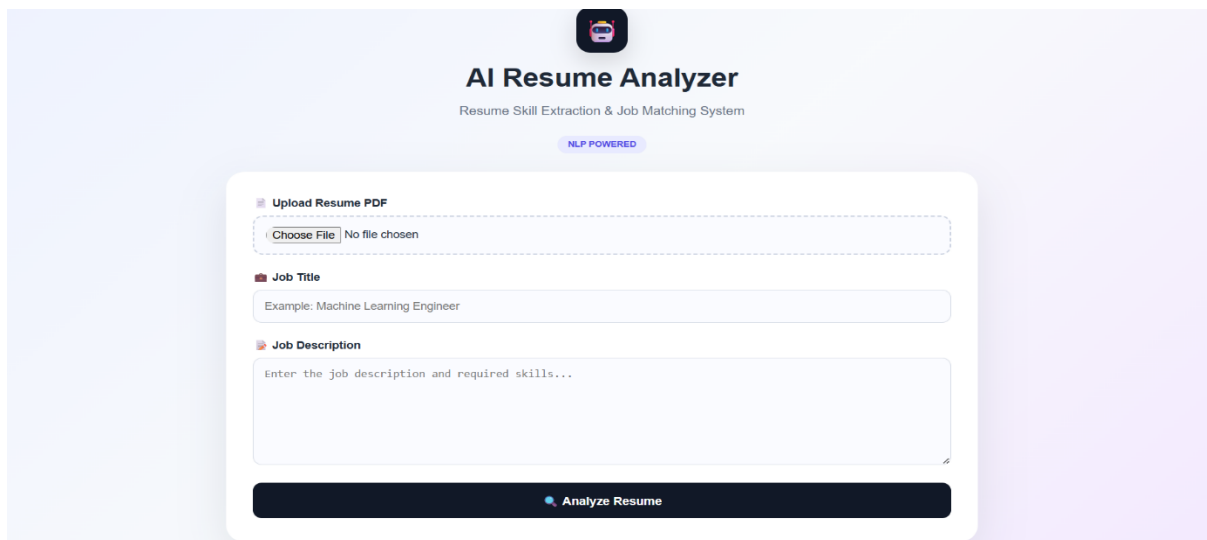
For the tested Machine Learning Engineer example:

Match Score: 31.85%

Skill Coverage: 54.55%

Skills Matched: 6 / 11

Flask Web Interface



The screenshot shows the 'AI Resume Analyzer' web interface. At the top, there is a logo of a robot head and the title 'AI Resume Analyzer' with the subtitle 'Resume Skill Extraction & Job Matching System'. Below this is a blue button labeled 'NLP POWERED'. The main form is white and contains three sections: 'Upload Resume PDF' with a 'Choose File' button and 'No file chosen' text; 'Job Title' with a text input field containing 'Example: Machine Learning Engineer'; and 'Job Description' with a larger text input field containing 'Enter the job description and required skills...'. At the bottom of the form is a dark blue button labeled 'Analyze Resume'.

17. Deployment

The Flask web application was deployed using Render.

Deployment configuration:

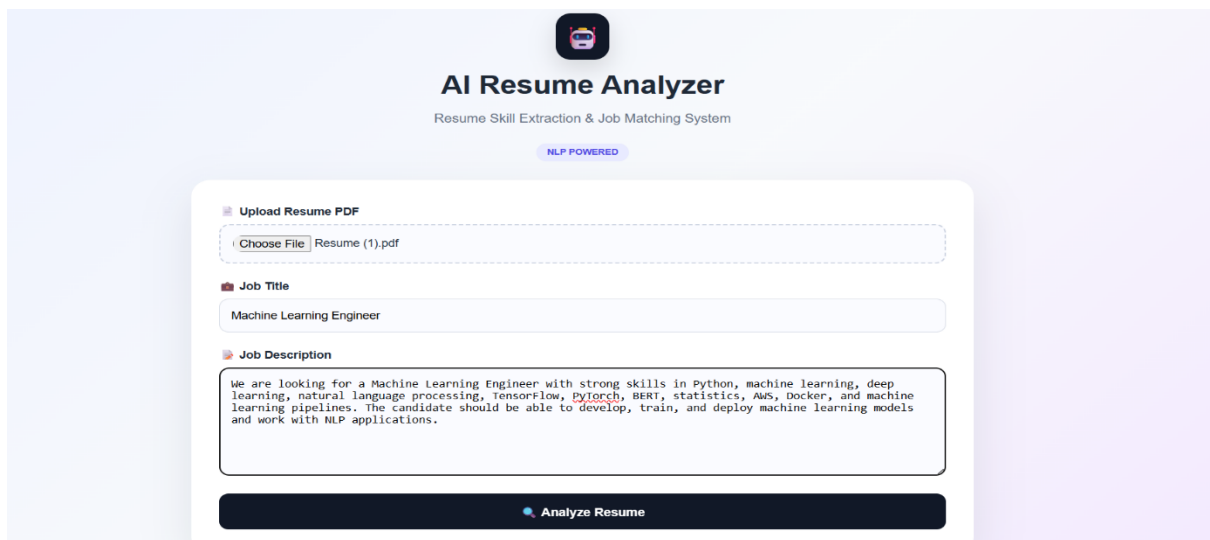
- Platform: Render
- Repository: GitHub
- Root Directory: Web_Interface
- Build Command: pip install -r requirements.txt
- Start Command: gunicorn app:app

The application is publicly accessible through:

Render Application:

<https://resume-skill-extraction-nlp.onrender.com>

Deployed Web Application



The screenshot displays the 'AI Resume Analyzer' web application. At the top, there is a logo of a robot head and the title 'AI Resume Analyzer' with the subtitle 'Resume Skill Extraction & Job Matching System'. A blue badge indicates 'NLP POWERED'. The main form is divided into three sections: 'Upload Resume PDF' with a 'Choose File' button and 'Resume (1).pdf' text; 'Job Title' with a text input field containing 'Machine Learning Engineer'; and 'Job Description' with a text area containing a job posting for a Machine Learning Engineer. At the bottom, there is a dark blue button labeled 'Analyze Resume'.

ANALYSIS COMPLETE

Machine Learning Engineer

Resume (1).pdf



3.12%

Match Score

0%

Skill Coverage

0

Skills Matched

0

Skills Extracted

✓ Matched Skills

No matching skills found.

✗ Missing Skills

No missing skills detected.

🔗 Extracted Resume Skills

18. GitHub Repository

The complete project source code, datasets, notebook, screenshots, and web interface files are maintained in GitHub.

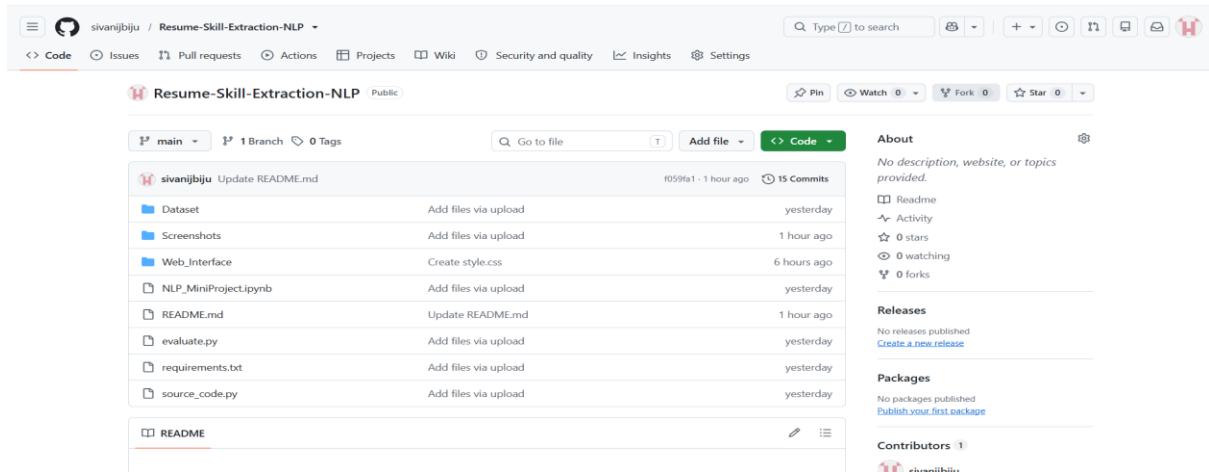
GitHub Repository:

<https://github.com/sivanijbiju/Resume-Skill-Extraction-NLP>

The repository contains:

- Dataset
- Screenshots
- Colab notebook
- Python source code
- Evaluation code
- Web interface
- Requirements
- README documentation

GitHub Repository



19. Advantages

The proposed system provides several advantages:

- Reduces manual resume screening effort.
- Automatically extracts technical skills.
- Provides quantitative matching scores.
- Ranks candidates based on job relevance.
- Shows missing skills.
- Supports PDF resume analysis.
- Provides a simple web interface.
- Can be extended to larger datasets.

20. Limitations

The current system has some limitations:

- Skill extraction depends on the predefined vocabulary.
- Synonyms and uncommon skill names may not always be detected.
- TF-IDF mainly considers word importance rather than deeper semantic meaning.

- The dataset is relatively small and based on sample data.
- Resume formatting and context may affect extraction.

21. Future Scope

The system can be improved by:

- Using advanced transformer models such as BERT.
- Adding Named Entity Recognition (NER).
- Supporting more resume formats.
- Using larger real-world datasets.
- Adding education and experience extraction.
- Providing personalized skill recommendations.
- Improving semantic similarity using modern NLP models.
- Adding recruiter dashboards and analytics.

22. Conclusion

The Resume Skill Extraction & Job Matching System demonstrates how NLP can be used to automate important parts of resume screening. The system performs text preprocessing, skill extraction, TF-IDF-based similarity calculation, job matching, candidate ranking, and skill comparison.

The evaluation achieved 100% Precision, 98.73% Recall, and 99.36% F1-score, showing strong skill extraction performance on the evaluation dataset.

A Flask web interface was also developed and deployed using Render, providing a practical way to upload and analyze resumes.